DisasterSight

AI-Assisted Satellite Damage Triage Dashboard for Rapid Post-Disaster Assessment

Proposal for Monash DeepNeuronx (MDN) probationary project

Team size: 5 members | Timeline: Weeks 9-12

Team members:

1. Matthew Chung Kai Shing (mchu0055@student.monash.edu)
2. Max Demunck (mdem0013@student.monash.edu)
3. Aaron Liu (aliu0064@student.monash.edu)
4. Evan Kong (ekon0016@student.monash.edu)
5. Ai Lin Tan (atan0221@student.monash.edu)

Executive summary

DisasterSight is our four-week AI/ML project proposal that **turns paired pre-disaster and post-disaster satellite imagery into a visual triage workflow**. The MVP will use **xBD building annotations** to extract building regions and train a **pre/post disaster damage classifier.** If time permits, we will add a lightweight building-localisation model to replace or supplement annotation-based localisation. The project is designed as a credible academic and industry-night prototype, not as a field-ready emergency response system. The team will use existing student resources, free/open-source tooling, and cached demo outputs where needed. If 4-class classification is unstable, we will collapse the task into 3 classes: no damage, damaged, destroyed, or 2 classes: no/low damage vs significant damage.

Decision area	Proposal choice
Core dataset	xBD / xView2 building-damage assessment
Core model	Two-stage pipeline: building localization + damage classification
Dashboard	Streamlit-first, local demo, optional FastAPI if time permits
Stretch goal	SpaceNet 8-inspired flooded-road/accessibility indicator
Budget	Use local machines, free notebooks, GitHub, and open-source tools
Claim level	Decision-support prototype for human review; no real deployment claims

1. Problem Statement

Post-disaster building assessment is time-critical, but manual review of large volumes of imagery is slow and difficult to scale. The xBD/xView2 benchmark exists because responders and analysts need faster ways to identify where buildings are damaged and how severe the damage is. For MDN, the opportunity is to build a compact and interpretable prototype that turns satellite imagery into a clear triage workflow for review and prioritization [1,2].

2. Proposed Solution

DisasterSight will provide an AI-assisted dashboard that ingests selected pre-disaster and post-disaster satellite image pairs, detects building locations, estimates building-level damage severity, aggregates results by zone, and presents the outputs in a stakeholder-friendly interface. The dashboard will include before/after imagery, color-coded damage overlays, summary statistics, ranked priority zones, and visible confidence or review flags. The system will be framed as decision support for human review, not automated dispatch or operational command.

Target user	Need	How DisasterSight helps
Emergency-management analysts	Rapid understanding of where damage is concentrated	Damage overlays, summary counts, and zone rankings
Humanitarian mapping teams	Clear visual evidence for response planning	Before/after viewer and interpretable building masks
Government/civil-protection reviewers	Transparent triage workflow	Explainable scoring logic and explicit limitations
Geospatial AI evaluators	Technical credibility	Benchmark-backed dataset, metrics, and repeatable pipeline

3. Dataset and Methodology

The MVP will use xBD/xView2 as the primary dataset because it directly supports paired pre/post building-damage assessment. xBD provides satellite imagery before and after disaster events, building polygons, damage labels, and metadata, making it the strongest fit for a four-week CNN/segmentation project [1,2]. SpaceNet 8 is relevant for flooded roads and access analysis, but it combines building footprint detection, road extraction, and flood detection across a broader remote-sensing task. Therefore, it should remain a stretch goal only [5,6].

Dataset	Use in proposal	Reason
xBD / xView2	Primary MVP dataset	Best match for paired pre/post building damage assessment
SpaceNet 8	Stretch/future extension	Useful for flooded roads and accessibility, but too broad for the core four-week scope
FloodNet / RescueNet / BRIGHT	Mention only as related work	Useful context, but less aligned with the exact MVP or too complex for the timeline

4. Technical Approach

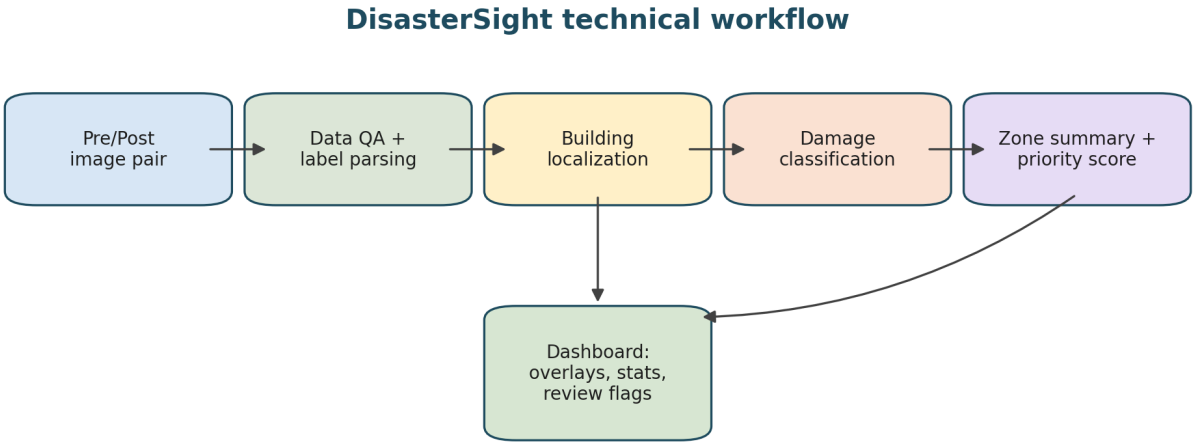

Figure 1. Proposed technical workflow for the xBD-based MVP.

The model pipeline will be kept deliberately simple and explainable. Stage 1 localizes buildings using a segmentation-style model such as U-Net with a pretrained encoder. Stage 2 classifies damage severity from paired pre/post building crops or masked image regions. This split is easier to debug and explain than a single model that tries to solve localization and severity at the same time. It also produces natural intermediate outputs for the dashboard: building masks, severity labels, confidence scores, and per-zone summaries [3,4].

Component	Recommended approach	Reason
Preprocessing	Pair pre/post images, parse polygons, rasterize masks, create building-centered crops	Converts raw labels into usable model inputs
Model	Two-stage segmentation + classification	Lower-risk and easier to inspect than a monolithic model
Training	Use pretrained encoders and a small event-aware subset	Keeps training feasible on free resources
Inference	Cache predictions for final demo scenes	Reduces live demo risk
Dashboard	Streamlit local app	Fastest way to build an interactive Python data app

5. MVP Scope

Use xBD building polygons to extract building regions, then train a pre/post image classifier to predict damage severity.

Scope level	Features
Must-have	- xBD dataset subset. - Pre/post image pair viewer. - Building crop extraction using xBD polygons. - Damage classifier using paired pre/post crops. - Dashboard with overlays, severity counts, priority score, and confidence flag. - Evaluation using macro F1, confusion matrix, and qualitative examples.
Should-have	- Simple building segmentation/localisation model. - Held-out disaster event testing. - Failure-case panel.
Stretch	- SpaceNet 8-style road/flood/accessibility indicator. - Human override notes. - FastAPI backend.
Avoid	- Real-time satellite data. - Route optimisation. - Full road network analysis. - Production deployment claims.

- Complex React frontend.
- Trying to solve all disaster types.

6. Dashboard Design

The dashboard WILL be designed for a two-minute industry-night walkthrough. The core experience should be simple: select a disaster scene, compare pre/post imagery, toggle damage overlays, inspect summary statistics, and review ranked zones. Streamlit is the preferred stack because it keeps the dashboard, model inference, and geospatial processing in Python [8]. FastAPI can be added only if a clean API boundary is needed [9].

Dashboard area	What it shows	Showcase value
Overview	Total buildings assessed, damage class counts, top priority zones	Gives stakeholders the answer quickly
Scene explorer	Synchronized before/after image view and color-coded overlays	Creates the main visual wow factor
Priority ranking	Zones ordered by severity and review status	Shows decision-support value
Model review	Confidence, failure cases, confusion matrix	Builds credibility and avoids overclaiming

7. Priority Scoring

The priority score should be simple, transparent, and clearly labelled as a demonstration score rather than a real emergency-response decision rule. Recommended formula:

Priority Score = 100 x (0.50 x Destroyed Share + 0.30 x Major Damage Share + 0.20 x Damage Density)

The dashboard should display the priority score separately from model confidence. If the model confidence is low or predictions are inconsistent, the zone should be marked 'Human review required' rather than automatically treated as high confidence. If the SpaceNet 8 stretch goal is achieved, an access penalty can be shown as an optional overlay, not a core promise.

8. Evaluation Plan

Evaluation area	Metric/output	Why it matters
Building localization	IoU, Dice, F1	Checks whether buildings are detected in the right place
Damage classification	Macro F1, precision, recall, confusion matrix	Handles class imbalance better than raw accuracy
Generalization	Held-out disaster/event examples	Shows honest performance beyond a random split
Dashboard usefulness	Qualitative walkthrough and failure-case review	Shows the product value, not just model scores
Demo reliability	Cached inference outputs and backup screenshots	Protects the industry-night presentation

9. Four-Week Delivery Plan

Week	Main goals	Deliverables	Backup plan
Week 9	Freeze scope, repo, dataset subset, data loader, dashboard shell	Proposal, repo structure, sample pipeline, clickable dashboard skeleton	Move all non-xBD features to stretch list
Week 10	Train first localization and damage baselines	Checkpoints, first overlays, confusion matrix	Use smaller subset and/or merge minor+major damage temporarily
Week 11	Integrate model outputs with dashboard	End-to-end prototype, zone summaries, confidence flags	Use cached predictions instead of live model inference
Week 12	Polish, rehearse, package final demo	Final dashboard, proposal, slides/poster, backup recording/screenshots	Use recorded walkthrough if live demo fails

10. Team Roles and Responsibilities

Role	Main responsibilities	Weekly outputs	Collaboration points
Data / preprocessing lead	xBD access, pairing, label parsing, rasterized masks, event-aware subset	Dataset README, train/val/test manifests, QA samples	Works with ML lead on input tensors and with dashboard lead on demo scenes
ML model lead	Training scripts, model architecture, checkpoints, inference pipeline	Baseline models, saved weights, inference script	Works with evaluation lead on metrics and integration lead on deployment format
Evaluation / experiments lead	Metrics, confusion matrix, qualitative review, failure-case analysis	Evaluation summary, plots/tables, limitation notes	Works with ML lead on experiments and product lead on talking points
Dashboard / frontend lead	Streamlit app, image viewer, overlays, summary cards, priority zones	Clickable dashboard, integrated UI, final visual polish	Works with data lead for scene assets and integration lead for demo flow
Integration / product lead	Scope control, interface contracts, proposal, demo story, final package	Product spec, integration checklist, final proposal, rehearsal plan	Coordinates all roles and owns the stakeholder narrative

Role	Difficulty	Why	Knowledge required
Data / preprocessing lead- Aaron	**Medium–Hard**	**This role is easy to underestimate. xBD data may have pre/post images, polygons, labels, and metadata that need to be cleaned into a usable ML format. If the data pipeline is messy, the whole project gets blocked.**	**Python, file handling, NumPy/Pandas, OpenCV/Pillow, JSON/GeoJSON, image resizing/cropping, train/validation/test split, basic geospatial concepts, rasterising polygons into masks**
ML model lead - Matt	**Hard**	**This is the most technically difficult role. They need to train or fine-tune the model, deal with class imbalance, poor accuracy, model checkpoints, and inference outputs.**	**CNNs, PyTorch or TensorFlow, transfer learning, ResNet/U-Net/Siamese CNN basics, training loops, loss functions, data loaders, augmentation, GPU/Colab, saving/loading models**
Evaluation / experiments lead - Ai Lin	**Medium**	**Not as hard as the ML lead, but very important. They must prove whether the model works and explain failures honestly. This person needs to understand metrics properly.**	**Accuracy, precision, recall, F1-score, macro F1, confusion matrix, IoU/Dice if segmentation is used, class imbalance, plotting with matplotlib/sklearn, interpreting model errors**
Dashboard / frontend lead - Evan	**Medium**	**Streamlit is beginner-friendly, but making the dashboard look polished and integrating overlays can be tricky. This role is very important for industry-night "wow factor."**	**Streamlit, Python, basic UI layout, image display, overlay visualisation, PIL/OpenCV, Pandas, caching outputs, basic UX design, colour legends, charts/summary cards**
Integration / product lead - Max Demunck	**Medium–Hard**	**Technically not the hardest, but coordination heavy. This person makes sure everyone's work fits together and the project tells a strong story. Bad integration can ruin a good ML project.**	**Git/GitHub, project management, documentation, interface contracts, proposal writing, presentation storytelling, risk control, demo planning, responsible AI framing**

11. Budget and Available Resources

The project will be scoped to run using existing student resources and free/open-source tooling. The team will avoid paid cloud commitments, paid hosting, and unnecessary production infrastructure. To manage compute limits, the team will train on a carefully selected subset, use pretrained encoders, cache predictions for demo scenes, and prepare a backup recorded walkthrough.

- **Python** — main development language
- **PyTorch** — CNN model training and inference
- **OpenCV / Pillow** — image preprocessing and visualisation
- **Rasterio + GeoPandas** — geospatial image and polygon handling
- **NumPy / Pandas** — data manipulation
- **scikit-learn** — metrics, confusion matrix, evaluation
- **Streamlit** — interactive dashboard
- **GitHub** — version control and collaboration
- **Google Colab / Kaggle Notebooks** — free GPU-based experimentation if available
- **Local machine demo with cached predictions** — no-cost, reliable industry-night showcase

Resource	Planned use	Budget impact
Local laptops/desktops	Coding, dashboard development, cached inference demo	AU$0
Google Colab / Kaggle notebooks	Free notebook-based training or experiments if needed	AU$0, subject to availability and limits [10,11]
GitHub	Code collaboration, issues, release branch	AU$0
Open-source Python stack	PyTorch, Streamlit, GeoPandas, Rasterio, scikit-learn	AU$0
Local Streamlit demo	Run dashboard on a team member's laptop for showcase	AU$0

12. Responsible AI and Limitations

DisasterSight must be presented as a human-in-the-loop academic prototype. False positives could misdirect attention, while false negatives could hide damaged areas. The system will therefore show confidence and review flags, document failure cases, and avoid claiming real operational deployment readiness. This aligns with NIST guidance on trustworthy AI characteristics such as validity, transparency, explainability, privacy enhancement, and bias management, as well as OCHA guidance on safe and ethical humanitarian data practices [12,13].

Principle	Proposal commitment	Demo implementation
Validity	Use event-aware evaluation and clear limitations	Metrics panel and held-out examples
Transparency	Describe scoring and model workflow	Visible legends, class counts, and score explanation
Human oversight	No autonomous response claims	Review-required labels for uncertain zones
Data responsibility	Use public historical data only	No live sensitive data ingestion
Generalization awareness	Acknowledge cross-disaster limits	Show at least one failure case professionally

13. Risk Management

Risk	Impact	Mitigation	Fallback
Dataset too large	Training and preprocessing delays	Use a small event-aware subset from Week 9	Restrict demo to one or two disasters
Poor 4-class damage accuracy	Weakens model credibility	Use class balancing and qualitative review	Merge minor+major damage for a simpler demo explanation
Dashboard integration late	No stable showcase	Build dashboard shell in Week 9	Use cached prediction outputs
Free compute unreliable	Training interruptions	Save checkpoints and reduce dataset size	Use local CPU/GPU for small inference and demo
Stretch goal scope creep	Core MVP diluted	Declare SpaceNet 8 as optional only	Cut stretch goal entirely
Live demo failure	Presentation risk	Rehearse and package backup assets	Use recording and screenshots

DisasterSight will use xBD pre/post disaster imagery to train a building-level damage classifier and display results in an interactive triage dashboard. Building localisation will initially use xBD annotations, with a segmentation model as a should-have or stretch feature.

One-Page Summary

Project title: DisasterSight: AI-Assisted Satellite Damage Triage Dashboard for Rapid Post-Disaster Assessment

Summary: DisasterSight converts paired pre-disaster and post-disaster satellite imagery into a visual triage workflow. The system will detect buildings, estimate damage severity, and present results through a dashboard designed for rapid human review. The project is scoped as a four-week, five-person MDN prototype with no budget request.

Problem: Manual post-disaster imagery review is slow and difficult to scale. DisasterSight addresses this by turning raw imagery into interpretable overlays, summary statistics, and priority-zone rankings for review.

Solution: The MVP uses xBD/xView2 as the primary dataset and implements a two-stage pipeline: building localization followed by damage classification. The dashboard will show before/after imagery, damage overlays, severity counts, a simple priority score, and low-confidence review flags.

Scope: Must-have features are xBD-based building damage assessment, dashboard overlays, summary statistics, and confidence/review flags. SpaceNet 8-style road accessibility is a stretch goal only. The team will avoid live satellite ingestion, route optimization, and operational deployment claims.

Timeline: Week 9: scope, data pipeline, dashboard shell. Week 10: baseline models and first metrics. Week 11: dashboard integration and priority scoring. Week 12: polish, rehearsal, final demo, backup assets.

Team: Roles are divided across data/preprocessing, ML modeling, evaluation/experiments, dashboard/frontend, and integration/product leadership.

Budget: AU$0 requested. The team will use local machines, GitHub, open-source Python tools, and free notebook resources where available. The final demo will use cached outputs if needed to avoid compute instability.

Responsible AI: DisasterSight is a decision-support prototype for human review, not an automated emergency-response system. It will show confidence, limitations, and failure cases.

References and Source Links

These links replace the broken bracket-style references from the draft. They should be kept as standard source links in the MDN proposal.

[1] SEI - Improving Disaster Response with the xView2 Challenge - Official xView2/xBD project overview.

[2] Gupta et al. - xBD: A Dataset for Assessing Building Damage from Satellite Imagery - Dataset paper for xBD.

[3] IBM - The xView2 AI Challenge - Industry write-up on xView2 modelling approach.

[4] Microsoft Research - Post-Disaster Building Damage Assessment Tools - Deployment-oriented discussion of damage assessment tools.

[5] Hansch et al. - SpaceNet 8: The Detection of Flooded Roads and Buildings - Paper summary for SpaceNet 8.

[6] SpaceNetChallenge/SpaceNet8 GitHub repository - Code and baseline materials for SpaceNet 8.

[7] Gerard et al. - A Simple, Strong Baseline for Building Damage Detection on xBD - Recent baseline and generalization discussion.

[8] Streamlit official site - Open-source Python framework for data and ML apps.

[9] FastAPI official documentation - Optional API layer if time permits.

[10] Google Colab - Free hosted notebooks; availability subject to limits.

[11] Kaggle Notebooks documentation - Notebook environment and accelerator options.

[12] NIST AI Risk Management Framework FAQ - Trustworthy AI and risk management guidance.

[13] OCHA Data Responsibility Guidelines 2025 - Humanitarian data responsibility guidance.